VERIQTA

SHELL SCRIPTING

PRODUCTION HANDBOOK

VOLUME 2

PRODUCTION FOCUSED AUTOMATION EXPERT LEVEL

AUTOMATE. ORCHESTRATE. OPTIMIZE. SCALE.

ADVANCED SCRIPTS
Write powerful and robust automation.

SYSTEM AUTOMATION
Manage servers, services, users, and more.

CLOUD & DEVOPS
Automate AWS, Azure, Docker, Kubernetes & CI/CD.

PRODUCTION READY
Build secure, reliable, and scalable automation.

REAL WORLD EXAMPLES
From monitoring to deployments and incident recovery.

/bin/bash

```bash
#!/bin/bash
# Production-Ready Automation
set -euo pipefail

LOG_DIR="/var/log/automation"
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

log() {
    echo "[$TIMESTAMP] $1" | tee -a "$LOG_DIR/script.log"
}
check_service() {
    if systemctl is-active --quiet "$1"; then
        log "✅ $1 is running"
    else
        log "❌ $1 is not running. Restarting..."
        systemctl restart "$1"
    fi
}
log "🚀 Script started"
check_service "nginx"
log "✅ All done"
```

- Real World Scripts
- Enterprise Examples
- Best Practices
- Troubleshooting
- Performance Tips

200+ ADVANCED EXAMPLES | MODULAR & REUSABLE | SECURE & RELIABLE | AUTOMATE EVERYTHING | PRODUCTION FOCUSED

FROM COMMAND LINE TO CLOUD AUTOMATION MASTERY.
BUILD SCRIPTS THAT WORK. KEEP SYSTEMS THAT RUN.

Instagram: @veriqta | LinkedIn: VERIQTA | X: @veriqta | YouTube: VERIQTA | VERIQTA

VERIQTA

1. Advanced Variables and Parameter Expansion

Write Smarter Scripts. Handle Real-World Scenarios.

Variables and parameter expansion make your scripts flexible, dynamic, and production-ready.
Master these. Your scripts become powerful.

1 Variable Expansion

Use $ to access the value of a variable.

```
name="Veriqta"
echo "Hello, $name!"      # Hello, Veriqta
```

2 Default Values

Use a default value if variable is not set.

```
: ${USER_NAME:=guest}
echo "User: $USER_NAME"      # guest
```

3 Assigning Defaults

Assign default only if variable is unset (not null).

```
: ${APP_ENV:=production}
echo "Environment: $APP_ENV"
```

4 Required Variables

Exit with an error if variable is not set.

```
: ${DB_PASSWORD:?Error: DB_PASSWORD
                        is required}"
echo "Connecting to DB..."
```

Script exits if DB_PASSWORD is not set.

5 Alternate Values

Use alternate value if variable is unset or null.

```
mode=${RUN_MODE:-dev}
echo "Mode: $mode"      # dev
```

6 Indirect Expansion

Get the value of a variable whose name is stored in another.

```
var_name="APP_NAME"
APP_NAME="Veriqta Shell Handbook"
echo "${!var_name}"     # Veriqta Shell Handbook
```

Useful for dynamic configs and frameworks.

7 Environment Variables

Use system or custom environment variables.

```
echo "Home: $HOME"
echo "Path: $PATH"
echo "Shell: $SHELL"
```

Use export to create environment variables.

```
export API_URL="https://api.example.com"
```

8 Production Examples

Real-world use in scripts and automation.

```
: ${LOG_DIR:=/var/log/myapp}
: ${TIMEOUT:=30}
: ${RETRY_COUNT:=3}
echo "Logs: $LOG_DIR | Timeout: $TIMEOUT |
      Retry: $RETRY_COUNT"
```

9 Best Practices

- ☑ Always quote your variables: "$var"
- ☑ Use defaults to make scripts resilient.
- ☑ Validate required variables early.
- ☑ Avoid exposing sensitive data in logs.
- ☑ Keep variable names meaningful and consistent.

Quick Reference Cheat Sheet

${var}	→ Variable value
${var:-default}	→ Use default if unset or null
${var:=default}	→ Assign default if unset
${var:?message}	→ Exit if unset or null
${var:+alternate}	→ Use alternate if set and not null
${!var}	→ Indirect expansion

Powerful scripts start with strong variable handling!

Remember

- Be explicit.
- Be secure.
- Be consistent.
- Be production-ready.

Write Once. Run Anywhere. Automate Everything.

VERIQTA

2. Advanced Arrays

Powerful Data Structures for Production-Ready Scripts.

Arrays help you store, organize, and manipulate multiple values efficiently.
Master arrays to write cleaner, smarter, and faster automation.
Think in collections. Automate with precision.

1 Indexed Arrays

Simple numeric index (starts at 0).

```
servers=("web01" "web02" "db01" "db02")
echo "${servers[0]}"        # web01
echo "${servers[2]}"        # db01
echo "${servers[@]}"        # all elements
echo "${#servers[@]}"       # length
```

2 Associative Arrays

Key-value pairs using named indexes.

```
declare -A user
user["name"]="Veriqta"
user["role"]="DevOps"
user["env"]="Production"
echo "${user[name]}"        # Veriqta
echo "${!user[@]}"          # all keys
```

3 Array Slicing

Extract a portion of an array.

```
nums=(1 2 3 4 5 6 7 8 9 10)
slice=("${nums[@]:2:4}")   # from
                           index 2, length 4
echo "${slice[@]}"         # 3 4 5 6
echo "${nums[@]:-3}"       # last 3
        elements -> 8 9 10
```

4 Adding Elements

Append or insert elements.

```
arr=("a" "b")
arr+=("c")                 # append
arr[2]="d"                 # set by index
arr=("${arr[@]}" "e" "f")  # append many
echo "${arr[@]}"           # a b d e f
```

5 Removing Elements

Remove by index or value.

```
arr=(a b c d e f)
unset 'arr[2]'             # remove index 2 (c)
echo "${arr[@]}"           # a b d e f
# remove by value (first match)
new_arr=()
for v in "${arr[@]}"; do
  [[ "$v" != "e" ]] && new_arr+=("$v")
done
echo "${new_arr[@]}"       # a b d f
```

6 Array Searching

Find elements using loops or built-ins.

```
arr=("api" "web" "db" "cache")
target="db"
for i in "${!arr[@]}"; do
  if [[ "${arr[$i]}" == "$target" ]]; then
    echo "Found at index: $i"
  fi
done
# Check existence (bash 4+)
[[ " ${arr[*]} " =~ " $target " ]] &&
 echo "Exists"
```

7 Nested Loops

Loop through multi-dimensional data.

```
users=("alice" "bob")
roles=("dev" "ops" "sec")
for u in "${users[@]}"; do
   for r in "${roles[@]}"; do
      echo "Assign $u -> $r"
   done
done
```

8 Production Examples

Real-world array usage.

- Read all files in a directory:
  `files=($(ls -1 /var/log/*.log))`
- Store server status:
  `declare -A status; status["web01"]="UP"`
- Process multiple services:
  `services=("nginx" "mysql" "redis")`
- Backup multiple databases:
  `dbs=("appdb" "userdb" "analyticsdb")`

9 Common Mistakes

- ✗ Using $arr instead of "${arr[@]}"
- ✗ Forgetting quotes → word splitting
- ✗ Unset index gaps → unexpective results
- ✗ Mixing indexed & associative syntax
- ✗ Not declaring associative arrays properly
- ✗ Modifying array while iterating

Bash arrays are powerful, but require precision!

Array Operations Cheat Sheet

Operation	Syntax	Example
Length	${#arr[@]}	echo "${#arr[@]}"
All Elements	${arr[@]}	echo "${arr[@]}"
First Element	${arr[0]}	echo "${arr[0]}"
Last Element	${arr[-1]}	echo "${arr[-1]}"
Slice	${arr:start:length}	echo "${arr[@]:1:3}"
All Keys (assoc)	${arr[@]}	echo "${!arr[@]}"
Check Key (assoc)	${arr[key]+yes}	[[-n "${arr[key]+}"]]

Array Mindset

- ☑ Store collections
- ☑ Access quickly
- ☑ Manipulate safely
- ☑ Automate reliably
- ☑ Think in arrays!

Best Practices

- ☑ Always quote array expansions.
- ☑ Use "${arr[@]}" for all elements.
- ☑ Use declare -A for associative arrays.
- ☑ Validate indexes before accessing.
- ☑ Keep arrays readable and consistent.
- ☑ Use meaningful names.
- ☑ Test with edge cases.
- ☑ Document array purpose.

Arrays turn complex data into simple automation.
Organize. Search. Process. Deliver.

3. Regular Expressions

Powerful Pattern Matching for Smarter Automation.

Regular Expressions (regex) help you search, validate, and extract patterns from text with precision.

Master regex. Automate Everything.

1 Regex Fundamentals

Regex is a sequence of characters that defines a search pattern.

Example:

```
echo "error 404 not found" | grep "error"
# Matches the word 'error'
```

Regex is supported by tools like grep, sed, awk, cut, tr, Perl, etc.

2 Character Classes

Match a set of characters.

```
[abc]       # a or b or c
[a-z]       # any lowercase letter
[A-Z]       # any uppercase letter
[0-9]       # any digit
[^abc]      # any char except a,b,c
[[:digit:]] # any digit (POSIX)
[[:alpha:]] # any letter (POSIX)
```

Example:

```
echo "abc123" | grep -E "[0-9]+"
# Matches: 123
```

3 Quantifiers

Define how many times a pattern can occur.

```
*       0 or more times
+       1 or more times
?       0 or 1 time
{n}     exactly n times
{n,}    n or more times
{n,m}   between n and m times
```

Example:

```
echo "aaab" | grep -E "a+"
# Matches: aaab
```

4 Anchors

Match positions in the text.

```
^     start of line
$     end of line
\b    word boundary
\B    non-word boundary
```

Example:

```
echo "ERROR: disk full" | grep -E "^ERROR"
# Matches lines that start with ERROR
```

5 Groups

Group parts of a pattern together.

```
(abc)      # capturing group
(?:abc)    # non-capturing group
|          # OR opperator
()         # group for precedence
```

Example:

```
echo "cat bat rat"| grep -E "(cat|bat)"
# Matches: cat bat
```

6 Capturing Patterns

Capture matched data for reuse.

```
(pattern)    # captured as \1
(pattern)    # next captured as \2
... up to \9
```

Example (sed backreference):

```
echo "2024-07-21" | sed -E 's/([0-9]{4})-
    ([0-9]{2})-([0-9]{2})/\\3-\2-\1/'
# Output: 21-07-2024
```

7 GREP with Regex

Use -E for extended regex.

```
grep -E "error" file.log          # simple match
grep -E "err(or|or)" file.log     # OR operator
grep -E "^[0-9]{4}-[0-9]{2}-[0-9]{2}" file.log
 # match date format
grep -E "[0-9]{3}-[0-9]{3}-[0-9]{4}" file.log
 # match phone number
grep -E -i "fail|failed|failure" file.log
 # case insensitive
```

8 Sed with Regex

Use regex for search and replace.

```
sed -E 's/error/ERROR/g' file.log                  # replace text
sed -E 's/[0-9]+/NUMBER/g' file.log                # digits -> NUMBER
sed -E 's/([0-9]{4})-([0-9]{2})-([0-9]{2})/\\3-\2-\1/' file.log
  # reorder date

                                                   # delete lines with ERROR
sed -E '/ERROR/d' file.log                         # print only
sed -n -E 's/.*(ERROR).*/\1/p' file.log              matched group
```

9 Production Log Parsing Examples

Real-world patterns used in log analysis and monitoring.

- Extract IP addresses:

```
grep -Eo '([0-9]{1,3}\\.){3}[0-9]{1,3}' access.log
```

- Extract timestamps (YYYY-MM-DD HH:MM:SS):

```
grep -Eo '[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}' app.log
```

- Extract HTTP status codes:

```
grep -Eo '"[0-9]{3}" access.log
```

- Count error lines:

```
grep -E -c "error|fail|exception" app.log
```

- Extract email addresses:

```
grep -Eo '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' users.log
```

10 Common Mistakes

- ✗ Forgetting to escape special characters.
- ✗ Using basic grep without -E.
- ✗ Wrong character class syntax.
- ✗ Greedy matching when not expected.
- ✗ Not anchoring patterns (slow searches).
- ✗ Confusing ( ) groups with [ ] classes.

11 Best Practices

- ✓ Test your regex with simple examples first.
- ✓ Use ^ and $ to anchor patterns.
- ✓ Prefer -E for readability.
- ✓ Use non-capturing groups (?:) when not needed.
- ✓ Keep regex simple and maintainable.

Regex gives your scripts X-ray vision. Use it wisely and your Automation will be Unstoppable.

VERIQTA

VERIQTA

4. Process Management

Control, Monitor, and Manage Processes Like a Pro.

Process management is key to building reliable, resilient, and production-ready automation. Know your processes. Control your system.

Observe. Control. Automate.

1 Background Jobs

Run commands in the background while your script continues.

```
long_task.sh &        # run in background
sleep 60 &            # another job
echo "Jobs started"
jobs                  # list background jobs
```

Use & to run tasks without blocking the script.

2 Foreground Jobs

Run commands in the foreground (default behavior).

```
echo "Starting backup..."
./backup.sh             # runs in
echo "Backup complete"  # foreground
```

The script waits for the command to finish before continuing.

3 Job Control

Manage jobs interactively.

```
Ctrl + Z        # suspend job
jobs            # list jobs
fg %1           # bring job 1
bg %1           # resume in
                  background
kill %1         # kill job 1
```

Works best in interactive shells, not in scripts.

4 Wait Command

Wait for background jobs to finish.

```
task1.sh &
pid1=$!
task2.sh &
pid2=$!
wait $pid1          # wait for
wait $pid2          # specific jobs
echo "All done"
```

wait with no PID waits for all background jobs.

5 Process IDs (PIDs)

Every process has a unique ID.

```
echo $$             # current shell PID
ps -p $$            # show info
pid=$(pgrep -f "nginx")
echo $pid           # store PID
ps -p $pid -o pid,cmd
```

Use PIDs to monitor, control, and kill processes.

6 Signals

Signals are used to communicate with processes.

SIGTERM	15	Terminate gracefully
SIGKILL	9	Kill immediately
SIGHUP	1	Reload configuration
SIGINT	2	Interrupt (Ctrl+C)
SIGSTOP	19	Stop process
SIGCONT	18	Continue process

Use the right signal for the right situation.

7 Killing Processes

Terminate processes safely.

```
kill -15 $pid           # polite (SIGTERM)
kill -9 $pid            # force (SIGKILL)
pkill -f "nginx"        # kill by name
killall -q nginx        # quiet killall
kill -TERM $(pgrep app)
```

Always try SIGTERM before SIGKILL.

8 Production Monitoring

Monitor processes in production.

```
top -b -n 1             # snapshot
ps aux --sort=%cpu      # top CPU
ps aux --sort=-%mem     # top MEM
pstat -a                # per-process stats
watch -n 5 'ps aux |
 grep myapp'            # watch output
```

Automate monitoring and alerting for critical processes.

9 Production Examples

Run a long backup in background and monitor:

```
backup.sh &
BACKUP_PID=$!
echo "Backup PID: $BACKUP_PID"
wait $BACKUP_PID
echo "Backup finished"
```

Restart service if not running:

```
if ! pgrep -x nginx > /dev/null; then
  echo "Nginx not running. Starting..."
  systemctl start nginx
fi
```

Monitor and kill high-CPU processes:

```
PID=$(ps -eo pid,pcpu--sort=-pcpu |
  awk 'NR==2{print $1!}')
CPU=$(ps -p $PID -o %cpu=)
if (( $(echo "$CPU > 90" | bc -l) )); then
  echo "High CPU: killing PID $PID"
  kill -15 $PID
fi
```

Log process start and exit status:

```
./deploy.sh &
PID=$!
echo "[$(date)] Deploy started PID=$PID" >>
      deploy.log
wait $PID
STATUS=$?
echo "[$(date)] Deploy finished status=$STATUS"
     >> deploy.log
```

10 Common Mistakes

- ✗ Forgetting & for background jobs.
- ✗ Not capturing PIDs correctly.
- ✗ Killing the wrong process.
- ✗ Using kill -9 first.
- ✗ Not checking if process exists.
- ✗ Not handling zombie processes.
- ✗ Ignoring exit codes.

Careless process management causes outages and data loss.

Best Practices

- ☑ Always log process actions.
- ☑ Use meaningful names.
- ☑ Handle errors and exit codes.
- ☑ Use SIGTERM before SIGKILL.
- ☑ Monitor critical processes.
- ☑ Clean up background jobs.
- ☑ Automate restarts and alerts.
- ☑ Document your process logic.

Process Management Cheat Sheet

Run in background	List jobs	Bring to foreground	Pause job	Wait for job	Kill politely	Kill forcefully
command &	jobs	fg %1	Ctrl + Z	wait $PID	kill -15 $PID	kill -9 $PID

A MASTER OF PROCESSES IS A MASTER OF PRODUCTION.

OBSERVE. CONTROL. AUTOMATE. DELIVER.

VERIQTA

5. Signal Handling

Handle Signals. Protect Processes. Stay in Control.

Signals are system events sent to processes to notify them of something important. Proper signal handling helps your scripts exit gracefully, clean up resources, and prevent data loss.

Handle signals like a professional engineer.

1 Common Linux Signals

Signal	Number	Name	Description	Default Action
SIGHUP	1	Hangup	Terminal closed	Terminate
SIGINT	2	Interrupt	Ctrl+C pressed	Terminate
SIGQUIT	3	Quit	Ctrl+\ pressed	Core dump
SIGTERM	15	Terminate	Termination request	Terminate
SIGKILL	9	Kill	Force kill (cannot be trapped)	Kill
SIGCONT	18	Continue	Resume if stopped	Continue
SIGSTOP	19	Stop	Pause process	Stop

2 SIGINT (Interrupt)

Triggered by: Ctrl + C

```
#!/bin/bash
trap 'echo "\n[!] Caught SIGINT. Exiting..."; exit 130' SIGINT
echo "Script running. Press Ctrl+C to stop."
while true; do
    echo "Working..."
    sleep 2
done
```

Exit code 130 = 128 + signal number (2)

3 SIGTERM (Terminate)

Triggered by: kill, system shutdown, service stop

```
#!/bin/bash
cleanup() {
    echo "[!] Caught SIGTERM. Cleaning up..."
    # pefrorm cleanup tasks
    exit 0
}
trap cleanup SIGTERM
echo "Script running. Send SIGTERM to stop."
while true; do sleep 5; done
```

4 SIGKILL (Kill)

Triggered by: kill -9 (cannot be caught or ignored)

- SIGKILL cannot be trapped, blocked, or ignored.
- It immediately terminates the process.
- Use only when a process is unresponsive.

kill -9 <PID>

5 Trap for Signals

trap command executes a function when a signal is received.

```
#!/bin/bash
cleanup() {
    echo "[!] Cleaning up before exit..."
    rm -f /tmp/mytempfile
    exit 0
}
trap cleanup SIGINT SIGTERM

echo "PID $$ running. Send SIGINT or SIGTERM to test."
while true; do sleep 3; done
```

Syntax

```
trap 'command' SIGNAL
trap 'function' SIGNAL

Multiple signals:
trap 'command' SIGINT
SIGTERM SIGHUP
```

6 Graceful Shutdown

Finish work safely before exiting.

```
#!/bin/bash
graceful_exit() {
    echo "[!] Graceful shutdown initiated..."
    echo "Stopping services..."
    sleep 2
    echo "Saving data..."
    sleep 2
    echo "Shutdown complete. Bye!"
    exit 0
}
trap graceful_exit SIGINT SIGTERM

# Main work
while true; do echo "Processing..."; sleep 5; done
```

7 Cleanup Routines

Always clean temporary files, locks, and resources.

```
#!/bin/bash
TMP_FILE="/tmp/mylock.$$"
touch "$TMP_FILE"
cleanup() {
    echo "[!] Removing lock file..."
    rm -f "$TMP_FILE"
}
trap cleanup EXIT SIGINT SIGTERM

echo "Lock file created: $TMP_FILE"
while true; do sleep 10; done
```

Exit Trap

EXIT runs when the script exits for any reason:

- ☑ Normal exit
- ☑ Ctrl+C
- ☑ kill
- ☑ Error (with exit)

8 Production Example: Service Watchdog

Auto-restart a service if it crashes.

```
#!/bin/bash
SERVICE="nginx"
restart_service() {
    echo "[!] Restarting $SERVICE..."
    systemctl restart $SERVICE
}
trap restart_service SIGTERM SIGINT
while true; do
   if ! systemctl is-active --quiet $SERVICE; then
      echo "[!] $SERVICE is down! Restarting..."
      restart_service
   fi
   sleep 10
done
```

9 Production Example: Log Rotation with Signals

Rotate logs on SIGHUP without stopping the service.

```
#!/bin/bash
LOGFILE="/var/log/myapp.log"
rotate_logs() {
    echo "[!] Rotating logs..."
    mv "$LOGFILE" "${LOGFILE}.old.$(date +%F-%T)"
    touch "$LOGFILE"
}
trap rotate_logs SIGHUP
echo "Send SIGHUP to $$ to rotate logs."
while true; do echo "Logging..." >> "$LOGFILE"; sleep 2; done
```

10 Best Practices

- ☑ Always handle SIGINT and SIGTERM for graceful exit.
- ☑ Perform cleanup: temp files, locks, DB connections, sockets.
- ☑ Use EXIT trap for final cleanup.
- ☑ Log signal events for observability.
- ☑ Never ignore signals silently.
- ☑ Do not use SIGKILL unless absolutely necessary.
- ☑ Test your signal handling in all environments.
- ☑ Make scripts idempotent during shutdown.

Reliable scripts handle signals like pros!

Great Engineers don't just write scripts. They build resilient, production-ready automation.

6. Advanced Input and Output

Powerful I/O Techniques for Production-Ready Scripts.

Master how your scripts read input and send output.
Control data flow. Handle errors. Build reliable automations.
Input + Output = Powerful Automation



1 Standard Input (STDIN)

Read input from the keyboard or another command.

```
# Read input from keyboard
read -p "Enter your name: " name
echo "Hello, $name"

# Read from another command
cat users.txt | while read line; do
   echo "User: $line"
done
```

2 Standard Output (STDOUT)

Normal output from a command or script.

```
echo "This is standard output"

ls -l /var/log
df -h | tee disk_usage.txt    # Save output and display
```

STDOUT is used for successful results and normal data.

3 Standard Error (STDERR)

Error messages and diagnostics.

```
echo "This is an error" 1>&2
ls /not/exist 2> error.log
command 2>&1 | tee all_output.log
```

Always separate errors from normal output.
Makes troubleshooting much easier.

4 Here Documents (<<)

Pass multi-line input to a command.

```
cat << EOF
Hello, Veriqta
This is a multi-line text.
It is sent to the cat command.
EOF
```

Great for configuration, templates, and long text blocks.

5 Here Strings (<<<)

Pass a single string as input to a command.

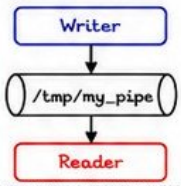
```
grep "error" <<< "$log_content"
wc -l <<< "one\ntwo\nthree"
```

Simple, clean, and perfect for small, single-line inputs.

6 Named Pipes (FIFO)

Create special files for communication between processes.

```
mkfifo /tmp/my_pipe
cat /tmp/my_pipe | while read line; do
  echo "Received: $line"
done &
echo "Hello FIFO" > /tmp/my_pipe
rm /tmp/my_pipe    # Clean up
```



Useful for inter-process communication and synchronization.

7 File Descriptors

Every open file or stream has a number.

FD	Description
0	Standard Input (STDIN)
1	Standard Output (STDOUT)
2	Standard Error (STDERR)
3+	Additional open files

```
# Redirecting with file descriptors
command > output.txt 1
command 2> error.txt 2
command > all.txt 2>&1
command < input.txt 0
```

8 Production Examples

```
# 1. Log errors to a file
./deploy.sh > deploy.log 2> deploy_error.log
# 2. Capture both output and error
./backup.sh > backup_all.log 2>&1
# 3. Real-time log monitor
tail -f /var/log/app.log 2>/dev/null
# 4. Read password securely
read -s -p "Enter password: " pass; echo
```

Key Principle: Control your input. Manage your output. Separate success from errors.

Great Scripts Handle Data Flow like Great Engineers.

7. Networking in Shell Scripts

Automate. Monitor. Integrate. Deliver.

Modern systems depend on network communication.
Shell scripts can interact with services, APIs, and endpoints to automate tasks and collect real-time data.

Network Automation = Reliable Automation

- ☑ FAST
- ☑ FLEXIBLE
- ☑ LIGHTWEIGHT
- ☑ POWERFUL

1 Curl - The Workhorse

Transfer data with URLs. Supports HTTP, HTTPS, FTP, and more.

```
# Get content from a URL
curl https://example.com
# Save output to a file
curl -o page.html https://example.com
# Show headers only
curl -I https://example.com
# Follow redirects
curl -L https://example.com
# POST data
curl -X POST -d "key=value" https://api.example.com/data
```

✅ Use curl for API calls, service checks, and data retrieval.

2 WGet - Simple Downloads

Easy file downloads and recursive retrieval.

```
# Download a file
wget https://example.com/file.zip
# Save with a specific name
wget -O file.zip https://example.com/file.zip
# Continue broken downloads
wget -c https://example.com/largefile.zip
# Download from a directory
wget -r -np -nH --cut-dirs=2 https://example.com/files/
```

✅ Use wget for bulk downloads and simple file automation.

3 Http Requests

Make HTTP requests and analyze responses.

```
# GET request with verbose output
curl -v https://example.com
# GET request with custom header
curl -H "User-Agent: Veriqta-Script" https://example.com
# POST request with JSON data
curl -X POST -H "Content-Type: application/json" \
  -d '{"name":"Veriqta","status":"active"}' \
   https://api.example.com/submit
# Save only the HTTP status code
curl -o /dev/null -s -w "%{http_code}" https://example.com
```

✅ Always check status codes to handle success and failure.

4 REST APIs

Interact with RESTful APIs in production.

```
# GET API
curl -s https://api.example.com/users
# GET with token
curl -H "Authorization: Bearer $TOKEN" \
  https://api.example.com/users
# POST data
curl -X POST -H "Content-Type: application/json" \
 -d '{"action":"create","name":"Veriqta"}' \
  https://api.example.com/items
# DELETE resource
curl -X DELETE -H "Authorization: Bearer $TOKEN" \
  https://api.example.com/items/123
```

✅ Secure your tokens. Never hardcode sensitive credentials.

5 Download Automation

Automate downloads and verify completeness.

```
# Download and verify file size
FILE_URL="https://example.com/bigfile.tar.gz"
wget -O /tmp/bigfile.tar.gz "$FILE_URL"
EXPECTED_SIZE=123456789
ACTUAL_SIZE=$(stat -c%s /tmp/bigfile.tar.gz)
if [ "$ACTUAL_SIZE" -eq "$EXPECTED_SIZE" ]; then
   echo "Download complete and verified"
else
   echo "Size mismatch! Download failed"
   exit 1
fi
```

✅ Always verify downloads to avoid corrupted files.

6 Connectivity Testing

Check network availability and reachability.

```
# Check if host is reachable
ping -c 3 example.com
# Check if port is open
nc -zv example.com 443    # netcat
# Check HTTP connectivity
curl -s --connect-timeout 5 https://example.com >/dev/null && \
  echo "Service is reachable" || echo "Service is down"
# DNS resolution test
nslookup example.com >/dev/null 2>&1 && echo "DNS OK" || echo "DNS FAIL"
```

✅ Use connectivity tests in health checks and monitoring.

7 Health Endpoints

Check service health using HTTP endpoints.

```
# Example health check
HEALTH_URL="https://api.example.com/health"
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$HEALTH_URL")
if [ "$STATUS" -eq 200 ]; then
   echo "Service is healthy"
else
   echo "Service unhealthy: HTTP $STATUS"
   exit 1
fi
```

✅ Health checks keep your systems reliable and self-healing.

8 Production Automation Example

Full example: Check API health and restart service if down.

```
API_URL="https://api.example.com/health"
SERVICE="myapp.service"
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$API_URL")
if [ "$STATUS" -ne 200 ]; then
   echo "API unhealthy (HTTP $STATUS). Restarting $SERVICE"
   sudo systemctl restart "$SERVICE"
   echo "Restart triggered at $(date)" >> /var/log/api_health.log
else
   echo "API healthy (HTTP $STATUS)"
fi
```

✅ Automate checks and recovery to ensure high availability.

Automate your network tasks. Monitor everything. Build reliable systems.
Connect. Check. Automate. Deliver.

VERIQTA

8. Working with JSON

Powerful JSON Processing with jq

JSON is a lightweight data format widely used in APIs and modern systems.
jq is a powerful command-line tool for querying and manipulating JSON.

jq turns JSON into actionable data.

1 jq Fundamentals

jq is a JSON processor that works on the command line.

```
$ jq '.field' file.json        # Get field
$ jq -r '.field' file.json     # Raw output
$ jq '.' file.json             # Pretty print whole JSON
```

2 Reading Json

Read JSON from a file or standard input.

```
$ jq . data.json
$ curl -s https://api.example.com/users | jq .
```

3 Extracting Values

Extract specific values or paths from JSON.

```
$ jq '.user.name' data.json          # Get name
$ jq '.users[] | .email' data.json   # Get all emails
$ jq '.items[0].price' data.json     # First item price
```

4 Updating JSON

Modify JSON data using jq filters.

```
$ jq '.user.active = true' data.json
$ jq '.users += [{"id":3,"name":"Charlie"}]' \
    data.json
$ jq 'del .obsolete' data.json       # Delete key
```

5 Formatting Output

Format JSON output for readability or compactness.

```
$ jq '.' data.json          # Pretty print
$ jq -c '.' data.json       # Compact output
$ jq -r '.name' data.json   # Raw text output
```

6 API Responses

Process JSON responses from web APIs.

```
$ curl -s https://api.example.com/data | jq '.data'
$ curl -s -H "Authorization: Bearer $TOKEN" \
  https://api.example.com/users | jq '.users[]| .name'
```

7 Production Examples

Real-world automation scenarios.

```
# Get user count from API
curl -s https://api.example.com/users | jq '. | length'
# Check if service is healthy
curl -s https://api.example.com/health | jq -e '.status == "ok"'
# Extract IDs of failed jobs
jq '.jobs[] | select(.status=="failed") | .id' jobs.json
```

8 Best Practices

- ✅ Use -r for raw output when needed
- ✅ Validate JSON before processing
- ✅ Handle errors in API responses
- ✅ Use clear and specific jq filters
- ✅ Combine jq with other tools for automation
- ✅ Keep queries simple and readable

JSON + jq = Powerful Data Processing and Automation
Parse. Extract. Transform. Automate.

VERIQTA

VERIQTA

9. System Monitoring Scripts

Automate Monitoring. Detect Issues. Stay Ahead.

Monitoring is not optional in production. These scripts collect real-time metrics, detect abnormal conditions, and trigger alerts.

See it. Understand it. Fix it before users do.

1 CPU Monitoring

- Check CPU usage and alert if high.
- Uses top and awk for analysis.

```
#!/bin/bash
CPU_USAGE=$(top -bn1 | awk '/Cpu/{print $2+$4}'
| sed 's/%us,//' | bc)
THRESHOLD=80
if (( $(echo "$CPU_USAGE > $THRESHOLD" | bc -l) )); then
  echo "[ALERT] High CPU Usage: ${CPU_USAGE}%"
fi
```

2 Memory Monitoring

- Check memory usage and available memory.
- Alert if usage exceeds threshold.

```
#!/bin/bash
MEM_USAGE=$(free -m | awk '/Mem:/ {printf "%.2f", $3/$2*100}')
THRESHOLD=80
if (( $(echo "$MEM_USAGE > $THRESHOLD" | bc -l) )); then
  echo "[ALERT] High Memory Usage: ${MEM_USAGE}%"
fi
```

3 Disk Usage Monitoring

- Monitor disk usage on all mounted filesystems.
- Alert if any filesystem is above threshold.

```
#!/bin/bash
THRESHOLD=85
df -hP | awk 'NR>1 {gsub("%", "", $5);
  if ($5 > th) printf "[ALERT] %s usage is %s%%\n",
  $6, $5}' th=$THRESHOLD
```

4 Network Monitoring

- Monitor network usage (bytes in/out).
- Detect high traffic or unusual spikes.

```
#!/bin/bash
INTERFACE=eth0
rx=$(cat /sys/class/net/$INTERFACE/statistics/rx_bytes)
tx=$(cat /sys/class/net/$INTERFACE/statistics/tx_bytes)
echo "RX: $((rx/1024/1024)) MB | TX: $((tx/1024/1024)) MB"
```

5 Service Health Check

- Check if critical services are running.
- Works with systemctl.

```
#!/bin/bash
SERVICE="nginx"
if systemctl is-active --quiet $SERVICE; then
  echo "[OK] $SERVICE is running"
else
  echo "[ALERT] $SERVICE is not running"
fi
```

6 Process Monitoring

- Monitor if important processes are running.
- Alert if process count is zero.

```
#!/bin/bash
PROCESS="sshd"
COUNT=$(pgrep -c $PROCESS)
if [ "$COUNT" -eq 0 ]; then
  echo "[ALERT] $PROCESS is not running"
else
  echo "[OK] $PROCESS is running ($COUNT)"
fi
```

7 Alert Generation

- Send alerts via email.
- Can be extended to Slack, Telegram, etc.

```
#!/bin/bash
ALERT_MSG="$1"
EMAIL="admin@example.com"
echo "$ALERT_MSG" | mail -s "[ALERT] Server Monitor" $EMAIL

# Example usage:
# ./alert.sh "Disk usage is 90% on /var"
```

8 Production Dashboard (Basic)

- Generate a simple HTML dashboard.
- View metrics in /opt browser.

```
#!/bin/bash
OUTFILE="/var/www/html/monitor.html"
cat > $OUTFILE <<EOF
<html><head><title>Server Monitor</title></head><body>
<h2>Server Monitoring Dashboard</h2>
<p><b>CPU Usage:</b> $CPU_USAGE%</p>
<p><b>Memory Usage:</b> $MEM_USAGE%</p>
<p><b>Disk Usage (/):</b> $(df -hP / | awk 'NR==2 {print $5}')</p>
<p><b>Uptime:</b> $(uptime -p)</p>
</body></html>EOF
echo "Dashboard generated at $OUTFILE"
```

Best Practices

- Run monitoring scripts at regular intervals using cron.
- Log all metrics and alerts to a file.
- Set proper thresholds based on your environment.
- Test your alerts and notifications regularly.
- Keep scripts simple, readable, and modular.
- Store scripts in /opt/monitoring/ or similar.
- Document your monitoring setup.

Cron Example

Run every 5 minutes

```
*/5 * * * * /opt/monitoring/
monitor.sh >> /var/log/monitor.log 2>&1
```

Monitor Everything That Matters. Alert Early. Solve Before It Breaks.
Good Monitoring = Stable Systems = Happy Users.

VERIQTA

10. User and Permission Management

Secure Access. Least Privilege. Automate. Validate.

Users and permissions are the foundation of system security.
Automate user setup. Enforce policies. Validate access.
Right Access. Right People. Right time.

1 User Creation

- Create standard users for daily access.
- Never use root for normal work.

```
# Create a new user
sudo useradd -m -s /bin/bash devuser
sudo passwd devuser

# Create with expiration date
sudo useradd -m -e 2024-12-31 analyst
```

2 Group Management

- Organize users with groups.
- Assign permissions to groups, not users.

```
# Create a group
sudo groupadd developers

# Add user to group
sudo usermod -aG developers devuser

# List group members
getent group developers
```

3 Password Policies

- Enforce strong passwords and aging.
- Use PAM and login.defs for policies.

```
# Set password to expire in 90 days
sudo chage -M 90 devuser

# Minimum days before change
sudo chage -m 7 devuser

# Check policy
sudo chage -l devuser
```

4 Sudo Configuration

- Grant admin rights safely.
- Use sudoers file via visudo.

```
# Edit sudoers safely
sudo visudo

# Example rule
devuser ALL=(ALL) NOPASSWD: /usr/bin/systemctl

# Validate syntax
sudo visudo -c
```

5 File Ownership

- Set correct owner and group.
- Prevents unauthorized changes.

```
# Change owner
sudo chown devuser:developers /opt/app

# Change owner recursively
sudo chown -R devuser:developers /opt/app

# Change group
sudo chgrp developers /opt/app
```

6 Permission Automation

- Automate permissions with scripts.
- Ensure consistency across servers.

```
#  Set directory permissions
sudo chmod 750 /opt/app

# Set file permissions
sudo find /opt/app -type f -exec chmod 640 {} \;

# Set directory permissions recursively
sudo find /opt/app -type d -exec chmod 750 {} \;
```

7 Security Validation

- Audit users and permissions regularly.
- Detect misconfigurations early.

```
# List all users
sut -d: -f1 /etc/passwd
# Check sudo users
getent group sudo
# Find world-writable files
sudo find / -xdev -type f -perm -0002
# Check SUID files
sudo find / -xdev -type f -perm -4000
```

8 Enterprise Practices

- Follow least privilege principle.
- Centralize identity and access.
- Log and review access regularly.

- ✅ Use LDAP/AD for centralized authentication
- ✅ Use roles and groups, not individual permissions
- ✅ Review access quarterly
- ✅ Remove unused accounts
- ✅ Enable logging: /var/log/secure or journald
- ✅ Use MFA for privileged access

Golden Rule

Give users the minimum access they need, for the minimum time they need it.

Permission Checklist

- ✅ Users created with home directory
- ✅ Strong password policies enforced
- ✅ Users in appropriate groups
- ✅ Sudo access is minimal and audited
- ✅ File ownership is correct
- ✅ Permissions follow least privilege
- ✅ Automated scripts are tested
- ✅ Access reviewed regularly

Security Warning

Weak permissions lead to breaches. Automate securely. Validate always.

Strong Access Control = Strong Security = Strong Systems

VERIQTA

11. Package Management Automation

Automate. Standardize. Keep Systems Updated. Stay Secure.



Package management keeps systems secure and consistent.
Automate installation, updates, and cleanup.
Automate Updates. Control Dependencies. Prevent Drift.



1 APT (Debian / Ubuntu)



- Advanced Package Tool
- Used on Debian, Ubuntu, and derivatives.

```
# Update package index
sudo apt update

# Install package
sudo apt install -y nginx

# Upgrade all packages
sudo apt upgrade -y

# Remove package
sudo apt remove -y nginx

# Clean up
sudo apt autoremove -y
sudo apt clean
```

2 YUM (RHEL / Centos 7)



- Yellowdog Updater Modified
- Used on RHEL 7, CentOS 7.

```
# Install package
sudo yum install -y httpd

# Update all packages
sudo yum update -y

# Remove package
sudo yum remove -y httpd

# Clean metadata
sudo yum clean all
sudo yum makecache
```

3 DNF (RHEL 8 / 9, Fedora)



- Dandified YUM
- Used on RHEL 8+, CentOS Stream, Fedora.

```
# Install package
sudo dnf install -y httpd

# Update all packages
sudo dnf upgrade -y

# Remove package
sudo dnf remove -y httpd

# Clean metadata
sudo dnf clean all
sudo dnf makecache
```

4 Zypper (Suse / Open Suse)



- ZYpp package manager
- Used on SUSE, openSUSE.

```
# Refresh repositories
sudo zypper refresh

# Install package
sudo zypper install -y apache2

# Update all packages
sudo zypper update -y

# Remove package
sudo zypper remove -y apache2

# Clean metadata
sudo zypper clean --all
```

5 Package Installation Automation



- Install multiple packages.
- Ensure packages are present.
- Run only if not already installed.

```
#!/bin/bash
PACKAGES=(nginx curl git jq htop)
for pkg in "${PACKAGES[@]}"; do
    if ! command -v "$pkg" &>/dev/null; then
        echo "Installing $pkg..."
        eudo apt install -y "$pkg"
    else
        echo "$pkg is already installed"
    fi
done
```

6 Automated Updates



- Keep systems patched.
- Automate updates with logging.

```
#!/bin/bash
LOGFILE="/var/log/auto-update.log"
echo "==== $(date) - Update Start =====" >> "$LOGFILE"
sudo apt update -y >> "$LOGFILE" 2>&1
sudo apt upgrade -y >> "$LOGFILE" 2>&1
echo "===== Update End =====" >> "$LOGFILE"
```

7 Dependency Management



- Resolve missing dependencies automatically.
- Verify package state.

```
# Debian/Ubuntu example
sudo apt install -f -y

# RHEL/CentOS example
sudo yum install -y --skip-broken

# DNF example
sudo dnf install -y --best --allowerasing
```

8 Production Automation Example



- Full automation script for production servers.

```
#!/bin/bash
set -euo pipefail

echo "Updating system and installing packages..."
sudo apt update -y
sudo apt upgrade -y
sudo apt install -y nginx curl jq htop
sudo systemctl enable --now nginx
echo "Packages installed and nginx started."
```

Best Practices

- [x] Always update package index before installing.
- [x] Use -y for non-interactive automation.
- [x] Log all actions for audit and troubleshooting.
- [x] Test scripts in staging before production.
- [x] Keep repositories clean and trusted.

Security Reminders



- [x] Apply security updates promptly.
- [x] Do not use --allow-unauthenticated.
- [x] Verify package sources.
- [x] Minimize unnecessary packages.

Common Check Commands

- # Check package version
  dpkg -l | grep <package>
  rpm -qa | grep <package>
- # Check installed size
  du -sh /var/cache/apt/archives
- # List outdated packages
  apt list --upgradable



Automate Packages. Eliminate Manual Work. Stay Secure. Stay Consistent.
Automation is Reliability.

VERIQTA

12. Service Management

Manage Services. Ensure Availability. Operate with Confidence.

systemd is the standard service manager on most modern Linux systems.
Use systemctl to control services, check status, and view logs.
Reliable Services = Reliable Systems = Reliable Business.

1 Systemctl Basics

- systemctl is used to manage systemd services.
- Works on most modern Linux distributions.

```
# Check systemd version
systemctl --version

# List all loaded units
systemctl list-units --type=service

# List all enabled services
systemctl list-unit-files --type=service --state=enabled
```

2 Service Status

- Check if a service is running.
- View detailed service information.

```
# Check status
systemctl status nginx
# Check if service is active (quick check)
systemctl is-active nginx
# Output: active / inactive / failed
# Check if service is enabled
systemctl is-enabled nginx
# Output: enabled / disabled
```

3 Starting Services

- Start a service manually.
- Verify after starting.

```
# Start a service
sudo systemctl start nginx
# Verify status
systemctl ŧiatus nginx
# Start and enable
sudo systemctl start nginx && sudo systemctl enable nginx
```

4 Stopping Services

- Stop a service safely.
- Confirm it is stopped.

```
# Stop a service
sudo systemctl stop nginx
# Verify
systemctl status nginx
# Force stop (if not stopping)
sudo systemctl kill nginx
```

5 Restarting Services

- Restart a service to apply changes.
- Useful after configuration updates.

```
# Restart a service
sudo systemctl restart nginx
# Reload configuration without full restart
sudo systemctl reload nginx
# Try reload, fallback to restart
sudo systemctl reload nginx || sudo systemctl restart nginx
```

6 Enable on Boot

- Enable service to start on system boot.
- Disable if not required.

```
# Enable service
sudo systemctl enable nginx
# Disable service
sudo systemctl disable nginx
# Check enablement
systemctl is-enabled nginx
```

7 Service Logs

- View service logs using journalctl.
- Troubleshoot issues from logs.

```
# View logs
sudo journalctl -u nginx
# Follow logs in real-time
sudo journalctl -u nginx -f
# Last 100 lines
sudo journalctl -u nginx -n 100
# Logs since boot
sudo journalctl -u nginx -b
```

8 Production Operations

- Use safe patterns in production.
- Monitor and alert on service failures.

```
# Check service and auto-restart if down
if ! systemctl is-active --quiet nginx; then
   echo "[ALERT] nginx is down. Restarting..."
   sudo systemctl restart nginx
fi

# Restart all critical services
for svc in nginx mysql redis; do
   sudo systemctl restart $svc
done
```

Best Practices

- ☑ Use systemctl for all service operations.
- ☑ Enable only required services.
- ☑ Monitor logs regularly.
- ☑ Set up alerts for critical services.
- ☑ Document service dependencies.

Common Pitfalls

- ☒ Forgetting to enable on boot.
- ☒ Ignoring failed services.
- ☒ Not checking logs for root cause.
- ☒ Using force kill instead of proper stop.
- ☒ Not monitoring service dependencies.

Common Systemctl Commands

```
systemctl start  <service>
systemctl stop   <service>
systemctl restart  <service>
systemctl status   <service>
systemctl enable   <service>
systemctl disable  <service>
systemctl is-active  <service>
systemctl list-units --type=service
```

Well-managed Services = Stable Systems = Happy Users = Successful Operations

VERIQTA

13. Database Automation

Automate. Protect Data. Ensure Integrity. Operate with Confidence.

Database automation reduces human error, ensures consistency, and protects data through scheduled tasks and validations.

Automate Backups. Validate Health. Maintain Reliability.

1 MySQL Scripts

- Automate MySQL administration tasks.
- Execute queries and maintenance.

```
# Connect to MySQL
mysql -u root -p

# Run SQL file
mysql -u root -p mydb < script.sql

# Execute single query
mysql -u root -p mydb -e "SELECT NOW();"
```

2 PostgreSQL Scripts

- Automate PostgreSQL administration.
- Execute SQL and maintenance tasks.

```
# Connect to PostgreSQL
psql -U postgres -d mydb

# Run SQL file
psql -U postgres -d mydb -f script.sql

# Execute single query
psql -U postgres -d mydb -c "SELECT NOW();"
```

3 Database Backups

- Create consistent backups automatically.
- Store backups with timestamps.

```
# MySQL backup
mysqldump -u root -p --all-databases \
  --routines --triggers \
  > /backups/mysql_$(date +%F_%H%M%S).sql

# PostgreSQL backup
pg_dump -U postgres -F custom -b -v -f \
  /backups/pg_$(date +%F_%H%M%S).backup mydb
```

4 Restore Automation

- Automate database restore processes.
- Ensure fast recovery.

```
# MySQL restore
mysql -u root -p mydb < /backups/mysql_backup.sql

# PostgreSQL restore
pg_restore -U postgres -d mydb \
  /backups/pg_backup.backup
```

5 Health Checks

- Verify database availability and performance.
- Alert on failures.

```
# MySQL health check
mysqladmin -u root -p ping | grep -q "alive"
if [ $? -ne 0 ]; then
  echo "[ALERT] MySQL is not responding!"
fi
# PostgreSQL health check
pg_isready -U postgres -d mydb || \
  echo "[ALERT] PostgreSQL is not responding!"
```

6 SQL Execution

- Run batch SQL statements.
- Use scripts for repeatable tasks.

```
# Execute multi-statement SQL file
mysql -u root -p mydb < /scripts/maintenance.sql

# Execute PostgreSQL script
psql -U postgres -d mydb -f /scripts/maintenance.sql

# Execute inline SQL
mysql -u root -p mydb -e "OPTIMIZE TABLE users;"
```

7 Production Maintenance

- Automate optimization and cleanup.
- Keep databases fast and clean.

```
# MySQL maintenance
mysql -u root -p -e "OPTIMIZE TABLE mytable;"
mysql -u root -p -e "ONALYZE TABLE mytable;"

# PostgreSQL maintenance
psql -U postgres -d mydb -c "VACUUM ANALYZE;"
psql -U postgres -d mydb -c "REINDEX DATABASE mydb;"
```

8 Best Practices

- Automate safely and securely.
- Test scripts before production.

- [x] Store credentials securely (use ~/.my.cnf or .pgpass)
- [x] Test backup and restore regularly
- [x] Verify backup integrity after preastion
- [x] Use compression for large backups
- [x] Set retention policy and cleanup old backups
- [x] Log all operations and monitor failures
- [x] Use least-privilege database accounts

Backup Retention Example

```
#!/bin/bash
BACKUP_DIR="/backups"
RETENTION_DAYS=14
find $BACKUP_DIR -type f -mtime +$RETENTION_DAYS
-name "*.sql" -o -name "*.backup" -exec rm -f {} \;
echo "Old backups deleted."
```

Sample Cron Jobs

```
# Daily MySQL backup at 2 AM
0 2 * * * /scripts/mysql_backup.sh

# Daily PostgreSQL backup at 3 AM
0 3 * * * /scripts/pg_backup.sh

# Weekly maintenance on Sunday 1 AM
0 1 * * 0 /scripts/db_maintenance.sh
```

Security Reminder

- Never hardcode passwords in scripts
- Use dedicated DB users for automation
- Restrict script access and permissions
- Encrypt sensitive backup data
- Monitor and audit database activities

Automated Databases = Reliable Systems = Protected Data = Happy Users

VERIQTA

14. Cloud Automation

Automate. Provision. Manage. Secure. At Scale.

Cloud automation eliminates manual work, reduces errors, and ensures consistent, scalable infrastructure.
Automate Infrastructure. Reduce Risk. Deliver Faster.

1 AWS CLI Scripting

- Automate AWS resources using AWS CLI.
- Ideal for provisioning, monitoring, and operations.

```
# Configure AWS CLI
aws configure

# List S3 buckets
aws s3 ls

# Create an EC2 instance
aws ec2 run-instances --image-id ami-12345678 \
 --instance-type t3.micro --key-name mykey

# Stop an EC2 instance
aws ec2 stop-instances --instance-ids i-0abcd12345
```

2 Azure CLI Scripting

- Automate Azure resources with Azure CLI.
- Great for enterprise cloud operations.

```
# Login to Azure
az login

# List resource groups
az group list -o table

# Create a resource group
az group create --name my-rg --location eastus

# Create a VM
az vm create --resource-group my-rg --name myvm \
 --image UbuntuLTS --admin-username azureuser --generate-ssh-keys
```

3 Google Cloud CLI

- Automate Google Cloud resources using gcloud.
- Scalable and powerful cloud operations.

```
# Login to Google Cloud
gcloud auth login
# Set project
gcloud config set project my-project
# List compute instances
gcloud compute instances list
# Create a VM instance
gcloud compute instances create my-vm \
--zone us-central1-a --machine-type e2-micro --image-family debian-11
```

4 Resource Automation

- Create, update, delete cloud resources.
- Use scripts for repeatable infrastructure.

```
# AWS - Create S3 bucket
aws s3 mb s3://my-unique-bucket-$(date +%s)

# Azure - Create Storage Account
az storage account create --name mystorage$RANDOM \
  --resource-group my-rg --location eastus --sku Standard_LRS

# GCP - Create Storage Bucket
gsutil mb -l us-central1 gs://my-bucket-$(date +%s)
```

5 Authentication & Credentials

- Secure authentication for CLI access.
- Use roles, profiles, and service accounts.

```
# AWS - Use named profile
aws configure --profile prod
# Azure - Service Principal Login
az login --service-principal -u <appId> -p <password> --tenant <tenantId>
# GCP - Service Account Login
gcloud auth activate-service-account --key-file key.json
# Export credentials (temporary)
export AWS_PROFILE=prod
```

6 Infrastructure Operations

- Monitor, modify, and manage infrastructure.
- Ensure availability and performance.

```
# AWS - Describe instances
aws ec2 describe-instances --query 'Reservations[].Instances[].
  InstanceId' --output text
# Azure - List VM status
az vm list --resource-group my-rg --show-details -o table
# GCP - Get instance metadata
gcloud compute instances describe my-vm --zone us-central1-a
```

7 Production Workflows

- Combine CLI commands into workflows.
- Use error handling and logging.

```
#!/bin/bash
set -euo pipefail
echo "[INFO] Starting AWS backup workflow..."
BUCKET="s3://my-backup-bucket"
aws s3 sync /data $BUCKET/$(date +%F)/
echo "[INFO] Backup completed successfully."
```

8 Production Automation Example

- Full automation example: Deploy + Monitor + Alert.
- End-to-end cloud automation.

```
#!/bin/bash
set -e
aws ec2 run-instances --image-id ami-12345678 --instance-type t3.micro \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=WebServer}]' \
  --output text > instance_id.txt
INSTANCE_ID=$(cat instance_id.txt)
echo "Launched instance: $INSTANCE_ID"
aws cloudwatch put-metric-alarm --alarm-name CPUHigh --metric-name CPUUtilization \
  --namespace AWS/EC2 --statistic Average --period 300 --comparison-operator GreaterThanThreshold
  --evaluation-periods 2 --alarm-actions arn:aws:sns:us-east-1:123456789012:alerts
```

Best Practices

- [x] Use least privilege IAM roles.
- [x] Store credentials securely.
- [x] Use environment variables or profiles.
- [x] Log all operations and errors.
- [x] Test scripts in non-production first.
- [x] Use retries and error handling.

Security Reminders

- [x] Never hardcode credentials.
- [x] Rotate keys and tokens regularly.
- [x] Use MFA and role-based access.
- [x] Restrict network access.
- [x] Encrypt sensitive data in transit and at rest.

Common AWS CLI Commands

```
aws s3 ls
aws ec2 describe-instances
aws iam list-users
aws cloudwatch list-metrics
aws sts get-caller-identity
```

Common Azure CLI Commands

```
az group list
az vm list
az storage account list
az monitor metrics list
az role assignment list
```

Automation is Power. Consistency is Reliability. Cloud is Freedom.

VERIQTA

15. Docker Automation

Automate. Containerize. Deploy. Monitor. Scale.

Docker automation ensures consistent, reliable, and repeatable container operations in production.

Automate Build. Run. Monitor. Clean. Repeat.

1 Running Containers

- Run containers in detached mode.
- Use environment variables and volumes.

```
# Run a container
docker run -d --name web \
  -p 8080:80 \\
  -e APP_ENV=production \\
  -v /opt/app/data:/data \
  nginx:latest

# List running containers
docker ps
```

2 Building Images

- Build custom images from Dockerfile.
- Tag images for versioning.

```
# Build image
docker build -t myapp:1.0.0 .

# Tag image
docker tag myapp:1.0.0 registry.example.com/myapp:1.0.0

# Push image
docker push registry.example.com/myapp:1.0.0
```

3 Container Cleanup

- Remove stopped containers, images, and volumes.
- Prevent disk space issues.

```
# Remove stopped containers
docker container prune -f

# Remove unused images
docker image prune -a -f

# Remove unused volumes
docker volume prune -f

# Full system cleanup
docker system prune -a --volumes -f
```

4 Docker Compose Automation

- Use docker-compose for multi-container apps.
- Automate up, down, and restart.

```
# Start services in background
docker compose up -d

# View running services
docker compose ps

# Restart a service
docker compose restart web

# Stop and remove services
docker compose down -v
```

5 Health Checks

- Check container health endpoints.
- Automate alerts on failure.

```
# Check container health status
docker inspect --format='{{.State.Health.Status}}' web

# HTTP health check inside container (example)
curl -f http://localhost:8080/health || echo "Unhealthy"

# Script health check for multiple containers
for c in $(docker ps --format '{{.Names}}'); do
 status=$(docker inspect --format='{{.State.Health.Status}}' "$c")
 echo "$c: $status"
done
```

6 Log Collection

- Collect and rotate container logs.
- Save logs for monitoring and audits.

```
# View logs (follow mode)
docker logs -f web

# Save logs to file
docker logs web > /var/log/containers/web.log 2>&1

# Limit log size (in json-file driver)
# /etc/docker/daemon.json
# { "log-driver": "json-file", "log-opts": { "max-size": "10m",
#    "max-file": "5" }
```

7 Production Automation

- Automate deploys, rollbacks, and monitoring.
- Use scripts for zero-touch operations.

```
#!/bin/bash
# Deploy new version
IMAGE_TAG=$1
docker pull registry.example.com/myapp:$IMAGE_TAG
docker stop -it <name
docker rm --ipp || true
docker run -d --name myapp -p 8080:80 \
 --restart unless-stopped \
 registry.example.com/myapp:$IMAGE_TAG
echo "Deployment complete: $IMAGE_TAG"
```

8 Best Practices

- Keep images small and secure.
- Use health checks and restart policies.
- Automate cleanup and log rotation.
- Monitor containers and set alerts.

- [x] Use official base images
- [x] Tag images with semantic versions (e.g. 1.0.0)
- [x] Scan images for vulnerabilities
- [x] Use --restart unless-stopped
- [x] Limit container resource usage
- [x] Centralize logs (e.g., ELK, Loki)
- [x] Automate backups and rollbacks

Docker Cheat Sheet

```
docker ps -a                  # List all containers
docker images                 # List images
docker inspect <name>         # Inspect container
docker logs -f <name>         # Follow logs
docker exec -it <name> bash # Enter container
docker stop <name>
docker rm <name>
docker rmi <image>
docker system df
docker network ls
```

Common Docker Commands

```
docker run                    # Run container
docker build                  # Build image
docker pull                   # Pull image
docker push                   # Push image
docker compose up -d          # Start services
docker compose down           # Stop services
```

Production Reminder

- [x] Always test in staging first
- [x] Automate cleanups and backups
- [x] Monitor health and resources
- [x] Keep Docker and images updated
- [x] Document your automation

Automation + Containers = Consistency + Reliability + Scale

16. Kubernetes Automation

Automate. Orchestrate. Observe. Recover. Scale.

Kubernetes automation reduces manual work, prevents configuration drift ensures reliable, repeatable deployments in production.

Automate Everyday Tasks. Ensure Reliability. Deliver Faster.

1 Kubectl Scripting

- Automate kubectl tasks with scripts.
- Interact with the Kubernetes API.

```
# Get all pods in all namespaces
kubectl get pods --all-namespaces -o wide

# Save output to a file
kubectl get nodes -o json > nodes.json

# Loop through namespaces
for ns in $(kubectl get ns -o jsonpath='{.items[*].metadata.name}'); do
  echo "Namespace: $ns"
  kubectl get pods -n $ns
done
```

2 Namespace Automation

- Create, delete, and manage namespaces.
- Enforce resource isolation.

```
# Create namespaces from a list
for ns in dev staging prod; do
  kubectl create namespace $ns
done

# Add labels
kubectl label namespace prod environment=production --overwrite

# Delete namespaces (except kube-system)
kubectl delete namespace $(kubectl get ns
-o jsonpath='{.items[?(@.metadata.name!="kube-system")].metadata.name}')
```

3 Deployment Automation

- Automate deployments and updates.
- Ensure zero-downtime releases.

```
# Apply deployment
kubectl apply -f deployment.yaml

# Set image with tag
kubectl set image deployment/myapp app=myapp:v2.0.0

# Scale deployment
kubectl scale deployment myapp --replicas=5

# Rollout status
kubectl rollout status deployment/myapp
```

4 Pod Monitoring

- Monitor pod health and status.
- Automate alert checks.

```
# Check pods not running
kubectl get pods --all-namespaces \
  --field-selector=status.phase!=Running

# Restart crashed pods in a namespace
for pod in $(kubectl get pods -n prod --field-selector=status.
 phase=Failed -o name); do
  echo "Restarting $pod"
  kubectl delete $pod -n prod
done

# Pod resource usage (metrics-server required)
kubectl top pods --all-namespaces
```

5 ConfigMaps Automation

- Create and update ConfigMaps.
- Inject configuration into workloads.

```
# Create configmap from file
kubectl create configmap app-config --from-file=./config/

# Create configmap from literals
kubectl create configmap app-env \
  --from-literal=ENV=production \
  --from-literal=LOG_LEVEL=info

# Update configmap (imperative)
kubectl create configmap app-config --from-file=./config/ --
 dry-run=client -o yaml | kubectl apply -f -
```

6 Secrets Automation

- Manage secrets securely.
- Automate creation and updates.

```
# Create secret from literal
kubectl create secret generic db-secret \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASS="StrongPass!123"

# Create secret from file
kubectl create secret generic tls-cert \
  --from-file=tls.crt \
  --from-file=tls.key

# Decode secret (base64)
kubectl get secret db-secret -o jsonpath='{.data.DB_PASS}' | base64 -d
```

7 Rollout Management

- Automate rollouts, rollbacks, and history.
- Ensure safe and controlled releases.

```
# Check rollout history
kubectl rollout history deployment/myapp

# Rollback to previous version
kubectl rollout undo deployment/myapp

# Rollback to specific revision
kubectl rollout undo deployment/myapp --to-revision=3

# Pause and resume rollout
kubectl rollout pause deployment/myapp
kubectl rollout resume deployment/myapp
```

8 Production Examples

- Real-world automation tasks.
- Save time and prevent human error.

```
# Daily cluster health report
kubectl get nodes,pods,svc,deploy --all-namespaces -o wide
  > /var/log/k8s-health-$(date +%F).log

# Auto-clean old completed jobs
kubectl delete job --all-namespaces \
  --field-selector=status.successful=1 \
  --older-than=168h

# Auto-scale deployments based on CPU (via HPA)
kubectl autoscale deployment myapp --cpu-percent=70 --min=2 --max=10
```

Best Practices

- ✅ Use namespaces to isolate environments.
- ✅ Store secrets securely (avoid hardcoding).
- ✅ Use version control for manifests.
- ✅ Use health checks and readiness probes.
- ✅ Automate backups and disaster recovery.

Common Pitfalls

- ❌ Hardcoding secrets in manifests.
- ❌ Deploying without resource requests/limits.
- ❌ Ignoring rollout status and failures.
- ❌ Not monitoring pod health.
- ❌ Deleting resources without backups.

Useful Kubectl Commands

```
kubectl get all -A
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> -f
kubectl exec -it <pod> -n <ns> -- /bin/bash
kubectl get events --sort-by=.lastTimestamp
```

Automate Kubernetes. Ensure Reliability. Scale with Confidence.

VERIQTA

17. CI/CD Automation

Automate. Build. Test. Deploy. Validate. Repeat.

CI/CD automation delivers code faster, safer, and more reliably.
It reduces manual work, prevents human error, and ensures consistency.
Automate Pipelines. Validate Early. Deploy Confidently.

1 Jenkins Scripting



- Automate jobs, builds, tests, and deployments.
- Use Jenkinsfile for pipeline as code.

```
# Jenkinsfile (Declarative Pipeline)
pipeline {
  agent any
  stages {
    stage('Build') { steps { sh 'mvn clean package' } }
    stage('Test') { steps { sh 'mvn test' } }
    stage('Deploy') { steps { sh 'ansible-playbook deploy.yml' } }
  }
  post { always { cleanWs() } }
}
```

2 GitHub Actions

- Automate workflows using GitHub Actions.
- Great for open source and cloud deployments.

```
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: mvn clean package
      - name: Deploy
        run: ./deploy.sh
```

3 GitLab CI

- Use .gitlab-ci.yml to define pipelines.
- Integrated with GitLab container registry.

```
# .gitlab-ci.yml
stages:
 - build
 - test
 - deploy
build:
  stage: build
  script: - mvn clean package
test:
  stage: test
  script: - mvn test
deploy:
  stage: deploy
  script: - kubectl apply -f k8s/
  only: - main
```

4 Azure DevOps

- Build, test, and deploy with YAML pipelines.
- Integrated with Azure services.

```
# azure-pipelines.yml
trigger:
 - main
pool:
   vmImage: 'ubuntu-latest'
steps:
 - task: Maven@3
   inputs:
     mavenPomFile: 'pom.xml'
     goals: 'clean package'

 - task: KubernetesManifest@0
   inputs:
     action: 'deploy'
     manifests: 'k8s/*.yaml'
```

5 Build Automation

- Compile code, resolve dependencies, package artifacts.
- Ensure reproducible and reliable builds.

```
# Maven Build
mvn clean package -DskipTests
# Gradle Build
./gradlew clean build
# NPM Build
npm ci && npm run build
# Docker Image Build
docker build -t myapp:${BUILD_ID} .
```

6 Deployment Automation

- Deploy to environments automatically.
- Use blue/green, rolling, or canary strategies.

```
# Deploy to Kubernetes
kubectl apply -f k8s/
kubectl set image deployment/myapp myapp=registry/myapp:${TAG}
# Rolling Update with status check
kubectl rollout status deployment/myapp --timeout=300s
# Helm Deployment
helm upgrade --install myapp ./chart -f values-prod.yaml
```

7 Validation Scripts

- Automate tests, linting, security scans, and compliance.
- Fail fast to prevent bad deployments.

```
# Run Tests
mvn test
# Lint Code
 flake8 .
# Security Scan
 trivy fs --exit-code 1 --severity HIGH,CRITICAL .
# Infrastructure Validation
  kubectl apply --dry-run=client -f k8s/
```

8 Enterprise Pipelines

- Multi-stage, multi-env, secure, and observable.
- Include approvals, scans, notifications, and rollbacks.

```
# Enterprise Pipeline Stages
1. Source -> 2. Build -> 3. Test -> 4. Security Scan ->
5. Package -> 6. Deploy (Dev) -> 7. Deploy (Staging) ->
8. Approval -> 9. Deploy (Prod) -> 10. Post-Deploy Tests ->
11. Notify -> 12. Monitor
# Features
- RBAC & Secrets Management
- Audit Logs & Notifications
- Rollback & Disaster Recovery Hooks
```

Best Practices

- [x] Use pipeline as code (Jenkinsfile, YAML).
- [x] Automate everything. No manual steps.
- [x] Run tests early and fail fast.
- [x] Scan for vulnerabilities and compliance.
- [x] Use secrets management (Vault, KMS).
- [x] Monitor pipelines and deployments.

Common Pitfalls

- [ ] Hardcoding credentials in scripts.
- [ ] Skipping tests or quality gates.
- [ ] Deploying without validation.
- [ ] No rollback or recovery strategy.
- [ ] Ignoring pipeline monitoring.

Key Tools

- Jenkins, GitHub Actions
- GitLab CI, Azure DevOps
- Maven, Gradle, NPM
- Docker, Kubernetes, Helm
- Ansible, Terraform, Helm
- SonarQube, Trivy, Snyk

Useful Commands

```
kubectl get pods -A
kubectl rollout status deployment/<name>
docker images
docker ps -a
helm list -A
aws s3 ls / aws sts get-caller-identity
```

Automated Pipelines = Faster Delivery = Higher Quality = Happy Users

18. Enterprise Shell Script Design

Design Once. Reuse Everywhere. Operate Safely. Scale Confidently.

Enterprise shell scripts are structured, reliable, secure, and maintainable. They follow standards, handle failures gracefully, and integrate with automation systems.

Build Standards. Enforce Reliability. Deliver Excellence.

1 Modular Architecture

- Break scripts into small, focused modules.
- Keep logic reusable and testable.

```
# main.sh
#!/bin/bash
set -euo pipefail

# Load libraries
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "$SCRIPT_DIR/lib/common.sh"
source "$SCRIPT_DIR/lib/validate.sh"
source "$SCRIPT_DIR/lib/deploy.sh"
main() {
    log_info "Starting deployment..."
    validate_environment
    deploy_application
    log_info "Deployment completed successfully."
}
main "$@"
```

2 Shared Libraries

- Centralize common functions in library files.
- Source libraries in every script.

```
# lib/common.sh
#!/bin/bash

log_info()    { echo -e "[INFO ] $(date '+%F %T') - $1"; }
log_warn()    { echo -e "[WARN ] $(date '+%F %T') - $1"; }
log_error()   { echo -e "[ERROR] $(date '+%F %T') - $1"; }

die()         { log_error "$1"; exit 1; }

run_cmd()  {
   local cmd="$@"
   log_info "Running: $cmd"
   eval "$cmd" || die "Command failed: $cmd"
}
```

3 Configuration Management

- Use config files for environment-specific values.
- Do not hardcode variables.

```
# config/default.conf
APP_NAME="myapp"
APP_VERSION="1.0.0"
DB_HOST="db.example.com"
DB_PORT="5432"
LOG_LEVEL="INFO"
ENVIRONMENT="default"

# Load configuration
CONFIG_FILE="${CONFIG_FILE:-config/default.conf}"
[[ -f "$CONFIG_FILE" ]] || die "Config file not found: $CONFIG_FILE"
source "$CONFIG_FILE"
```

4 Environment Separation

- Separate configurations and resources by environment.
- Prevent cross-environment impact.

```
# config/prod.conf
APP_NAME="myapp"
DB_HOST="db.prod.example.com"
DB_PORT="5432"
ENVIRONMENT="production"
LOG_LEVEL="WARN"
REPLICAS="5"

# Select environment
ENV="${ENVIRONMENT:-dev}"
export CONFIG_FILE="config/${ENV}.conf"
```

5 Logging Framework

- Consistent, structured logging.
- Log to file and console.

```
# lib/logger.sh
init_logger() {
   LOG_DIR="${LOG_DIR:-/var/log/enterprise-scripts}"
   mkdir -p "$LOG_DIR"
   LOG_FILE="$LOG_DIR/$(basename "$0").log"
   touch "$LOG_FILE"
}
log() {
   local level="$1"; shift
   local msg="$@"
   local line="[$(date '+%F %T')] [$level] $msg"
   echo "$line" | tee -a "$LOG_FILE"
}
```

6 Error Framework

- Handle errors predictably.
- Cleanup and exit with meaningful messages.

```
# lib/error.sh
set -euo pipefail
trap 'on_error $LINENO $BASH_COMMAND' ERR
trap cleanup EXIT INT TERM
on_error() {
   local line=$1
   local cmd=$2
   log_error "Error on line $line: $cmd"
   exit 1
}
cleanup() {
   log_info "Performing cleanup..."
   # cleanup tasks here
}
```

7 Reusable Components

- Build generic functions and templates.
- Avoid duplication.

```
# lib/validate.sh
validate_required() {
   local var_name="$1"
   local value="${!var_name:-}"
   [[ -z "$value" ]] && die "Required variable '$var_name' is not set."
}
validate_port() {
   local port="$1"
   [[ "$port" =~ ^[0-9]+$ ]] || die "Port must be numeric."
   (( port > 0 && port < 65536 )) || die "Invalid port: $port"
}
```

8 Production Standards

- Follow conventions and security best practices.
- Ensure scripts are safe, readable, and maintainable.

- [x] Use: set -euo pipefail
- [x] Quote variables: "$VAR"
- [x] Validate inputs and environment
- [x] Use functions and libraries
- [x] Avoid global variables
- [x] Use structured logging
- [x] Handle errors and cleanup
- [x] Handle error flag and cleanup
- [x] Use meaningful exit codes
- [x] Minimize external dependencies
- [x] Code review and documentation
- [x] Test in staging before production

Common Directory Structure

```
enterprise-scripts/
├── bin/          # Executable scripts
├── lib/          # Shared libraries
├── config/       # Environment configs
├── templates/    # Reusable templates
├── logs/         # Log directory
└── README.md     # Documentation
```

Script Template (Skeleton)

```
#!/bin/bash
# Script: <name>
# Description: <purpose>
# Author: <team>
# Date: <date>
set -euo pipefail
# Load libraries
# Main logic
```

Useful Commands

bash -n script.sh	# Syntax check
shellcheck script.sh	# Static analysis
shfmt -w script.sh	# Format code
chmod +x script.sh	# Make executable
./script.sh --help	# Help output

Production Reminder

- [ ] Always test scripts in staging first.
- [ ] Protect sensitive data and secrets.
- [ ] Monitor script execution.
- [ ] Alert on failures.
- [ ] Automate with confidence.

Great Scripts Aren't just written. They're Designed, Tested, and Trusted.

VERIQTA

19. Production Case Studies

Real Problems. Real Automation. Real Impact.

These case studies show how automation solves real production problems, improves reliability, and saves time at scale.

Automate Repeated Tasks. Prevent Incidents. Recover Faster.

1 Automated Server Provisioning

- Provision consistent servers.
- Reduce manual setup time.
- Enforce security baseline.

Key Script/Command:

```
#!/bin/bash
# create_user, install packages, configure
useradd -m -s /bin/bash deploy
echo "deploy:StrongPass123" | chpasswd
yum install -y epel-release
yum install -y nginx git htop
systemctl enable nginx --now
ufw allow 'Nginx Full'
echo "Provisioning completed."
```

✅ Result: 15 min manual work → 2 min automated.

2 Automated Backup Framework

- Automated database backups.
- Compression and retention.
- Offsite storage to S3.

Key Script/Command:

```
#!/bin/bash
DATE=$(date +%F_%H%M)
mysqldump -u root -p$DB_PASS $DB_NAME \
  | gzip > /backups/db_$DATE.sql.gz
aws s3 cp /backups/db_$DATE.sql.gz \
  s3://my-backups/db/$DATE.sql.gz
find /backups -type f -mtime +7 -delete
echo "Backup completed: $DATE"
```

✅ Result: Zero data loss. Fully automated.

3 Log Rotation Automation

- Prevent disk space issues.
- Compress and archive logs.
- Keep systems healthy.

Key Script/Command:

```
#!/bin/bash
LOG_DIR="/var/log/myapp"
ARCHIVE="/var/log/archive"
mkdir -p $ARCHIVE
find $LOG_DIR -type f -name "*.log" -mtime +7 \
  -exec gzip {} \;
find $LOG_DIR -type f -name "*.log.gz" -mtime +30 \
  -exec mv {} $ARCHIVE/ \;
echo "Log rotation completed."
```

✅ Result: Prevented outages due to full disk.

4 API Monitoring Automation

- Monitor API availability.
- Alert on failures.
- Store metrics for analysis.

Key Script/Command:

```
#!/bin/bash
URL="https://api.myapp.com/health"
STATUS=$(curl -s -o /dev/null -w "%{http_code}" $URL)
if [ "$STATUS" -ne 200 ]; then
  echo "$(date): API DOWN (Status: $STATUS)" \
    | mail -s "API ALERT" ops@mycompany.com
else
  echo "$(date): API UP"
fi
```

✅ Result: Issues detected instantly, before users.

5 Kubernetes Automation

- Deploy apps consistently.
- Auto-scale and self-heal.
- Rollbacks with one command.

Key Script/Command:

```
#!/bin/bash
NAMESPACE=production
APP=myapp
IMAGE=myapp:$1
kubectl create namespace $NAMESPACE --dry-run=client -o yaml | \
kubectl apply -f -
kubectl set image deployment/$APP $APP=$IMAGE -n $NAMESPACE
kubectl rollout status deployment/$APP -n NAMESPACE
  --timeout=300s
echo "Deployment of $IMAGE completed."
```

✅ Result: Faster deployments. Fewer failed releases.

6 Incident Recovery Automation

- Detect failure.
- Auto-remediate.
- Escalate if unresolved.

Key Script/Command:

```
#!/bin/bash
SERVICE=nginx
systemctl is-active --quiet $SERVICE
if [ $? -ne 0 ]; then
  echo "$(date): $SERVICE down. Restarting..."
  systemctl restart $SERVICE
  sleep 5
  systemctl is-active --quiet $SERVICE || \
   mail -s "$SERVICE still down!" ops@company.com
fi
```

✅ Result: Reduced MTTR from hours to minutes.

7 Senior Engineer Lessons

- ✅ Automate everything repeatable.
- ✅ Make scripts idempotent.
- ✅ Validate inputs and environment.
- ✅ Fail fast and fail safe.
- ✅ Observe and log everything.
- ✅ Design for scale and change.
- ✅ Security and least privilege always.
- ✅ Test in staging, then production.

8 Best Practices

- ✅ Use version control for scripts.
- ✅ Standardize naming and structure.
- ✅ Keep secrets out of scripts.
- ✅ Use environment variables.
- ✅ Centralize logs and monitoring.
- ✅ Document usage and dependencies.
- ✅ Schedule with cron or systemd timers.
- ✅ Continuously review and improve.

9 Production Impact

- ✅ Higher availability.
- ✅ Fewer human errors.
- ✅ Consistent environments.
- ✅ Faster deployments.
- ✅ Better security posture.
- ✅ Lower operational cost.
- ✅ More time for engineering.
- ✅ Happy users.

Common Utility Functions

```
log_info()  { echo -e "[INFO] $1"; }
log_warn()  { echo -e "[WARN] $1";}
log_error() { echo -e "[ERROR] $1";}
die()       { log_error "$1"; exit 1; }
run_cmd()   { log_info "Running: $*"; "$@"; }
```

Standard Header Template

```
#!/bin/bash
# Script: <name.sh>
# Purpose: <what it does>
# Author: <your name>
# Date: $(date +%F)
# Version: 1.0
```

Cron Example

```
# Edit crontab
crontab -e
# Run backup daily at 2 AM
0 2 * * * /opt/scripts/backup.sh
# Run log rotation daily at 3 AM
0 3 * * * /opt/scripts/logrotate.sh
```

Systemd Timer Example

```
my-script.service
my-script.timer

# Enable & start
systemctl enable --now my-script.timer
```

Automation isn't just code. It's Reliability, Confidence, and Control at Scale.

VERIQTA

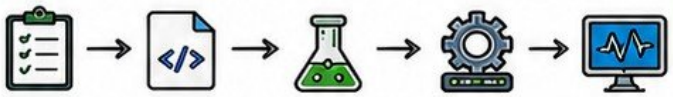
20. Volume 2 Production Review

Automate Smarter. Operate Safer. Deliver Consistently.

Volume 2 takes your shell scripting skills into real production environments. You now build automation that is reliable, secure, observable, scalable, and ready for enterprise operations.

Consistency. Reliability. Automation. Excellence.

1 Enterprise Shell Scripting Workflow



1. Plan	2. Code	3. Test	4. Deploy	5. Operate
• Define problem	• Modular design	• Unit testing	• Version control	• Monitor
• Identify inputs	• Functions	• Edge cases	• Permissions	• Maintain
• Design flow	• Logging	• Dry runs	• Environment	• Optimize
• Risk assessment	• Error handling	• Validation	• Scheduling	• Improve

2 Advanced Command Cheat Sheet

Category	Command	Purpose	Example
Text Processing	awk	Pattern scanning	awk "{print $1}" file.txt
Text Processing	sed	Stream editing	sed "s/old/new/g" file.txt
File Search	find	Find files	find / -type f -name "*.log"
Parallel Exec	xargs -P	Parallel execution	ls | xargs -P 4 -I {} rm {}
Network	curl	HTTP requests	curl -s -o /dev/null -w "%{http_code}" url
JSON	jq	Parse JSON	jq ".items[] .name" data.json
Archive	tar	Create archives	tar -czf backup.tgz /data
Checksum	sha256sum	File integrity	sha256sum file.bin
Disk Usage	du -sh	Human readable size	du -sh /var/log
Process	ps aux	List processes	ps aux | grep nginx
Permissions	chmod	Change permissions	chmod 750 script.sh
Scheduling	cron	Job scheduling	0 2 * * * /script.sh

Automation Checklist

- [x] Script is modular and readable
- [x] All inputs are validated
- [x] Error handling is implemented
- [x] Logging is implemented
- [x] Dry-run / test mode available
- [x] Script works in target environment
- [x] Dependencies are documented
- [x] Resource cleanup is in place
- [x] Script is idempotent
- [x] Scheduled jobs are monitored

Security Checklist

- [x] No hardcoded passwords or secrets
- [x] Sensitive data stored in files with 600 perms
- [x] Least privilege used
- [x] Input validation to prevent injection
- [x] Proper file permissions (700/750)
- [x] Scripts run as dedicated service users
- [x] Secrets externalized (env/secret stores)
- [x] Audit logging enabled
- [x] Cron jobs use absolute paths
- [x] Regular security review performed

Troubleshooting Checklist

- [x] Check script logs first
- [x] Verify input parameters
- [x] Check permissions and ownership
- [x] Validate external dependencies
- [x] Reproduce with debug mode (set -x)
- [x] Check system resources (CPU, RAM, Disk)
- [x] Verify network connectivity
- [x] Check exit codes
- [x] Review recent changes
- [x] Escalate with full context

Performance Checklist

- [x] Avoid unnecessary loops
- [x] Use batch operations
- [x] Minimize subshells and forks
- [x] Use grep/awk/sed efficiently
- [x] Check disk IO and memory usage
- [x] Parallelize where safe (xargs -P)
- [x] Cache expensive operations
- [x] Avoid parsing large outputs
- [x] Monitor execution time
- [x] Continuously profile and optimize

Senior Engineer Insights

- ★ A good script works.
- ★ A great script works for years.
- ★ Write for the operator who will run it at 2 AM.
- ★ Automate the boring.
- ★ Standardize everything.
- ★ Observability is not optional.
- ★ Treat scripts as production code.
- ★ Assume failure. Design for recovery.
- ★ Simplicity today prevents outages tomorrow.

Production Readiness Review

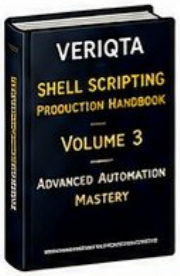
Is your automation...

- [x] Reliable?
- [x] Repeatable?
- [x] Secure?
- [x] Observable?
- [x] Maintainable?
- [x] Scalable?
- [x] Recoverable?
- [x] Production ready?

APPROVED

- Ready for scale
- Ready for reliability
- Ready for production

9 Preview of Volume 3 – Advanced Automation Mastery

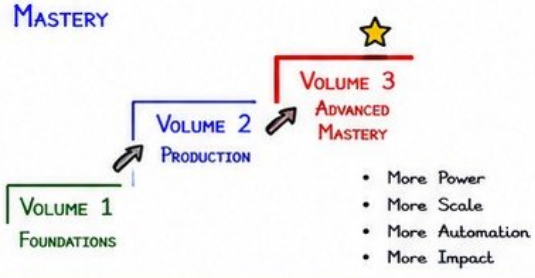


- ➡ Advanced orchestration patterns
- ➡ Event-driven automation
- ➡ Advanced error handling strategies
- ➡ Large-scale automation frameworks
- ➡ API integrations at scale
- ➡ Advanced logging & observability
- ➡ Performance tuning at scale
- ➡ Enterprise automation blueprints
- ➡ Real-world advanced case studies
- ➡ Building your automation platform

You've built the foundation. Now it's time to master advanced automation and enterprise-scale engineering.

Level Up.

Volume 3 is coming!



Automation isn't just a Skill. It's an Engineer's Superpower.